BDA Week 1 — Introduction to Big Data Analytics (The Big Picture)

Foundation module · Lightest topic · Conceptual + a few small numerical traces

BDA Final Exam Prep · Comprehensive Tutor

- Week 01 · Introduction to Big Data Analytics
  - §1 — Beginning Key Notes (Study Compass)
  - §2 — What is Big Data? The Five V's
    - V1 — Volume (Scale of Data)
    - V2 — Velocity (Speed of Data)
    - V3 — Variety (Diversity of Sources)
    - V4 — Veracity (Uncertainty of Data)
    - V5 — Value (Usefulness for Decisions)
    - Comparative table — V's vs traditional system limitations
    - Why exactly does Excel / RDBMS fail?
  - §3 — Bonferroni's Principle
    - The principle, stated precisely
    - Worked example — the "TIA" terrorist-pattern example (MMDS §1.2.3)
  - §4 — Distributed Computing: Why a Single Machine Cannot Cope
    - The four bottlenecks (course slide form)
    - The Big Data philosophy (the "at-a-glance" slide)
  - §5 — Hadoop Distributed File System (HDFS)
    - Core concepts
    - Mini architecture diagram (ASCII)
    - Storage cost
  - §6 — MapReduce Programming Model
    - The three phases
    - Diagram (ASCII)
    - Canonical example — word count
  - §7 — Other Tools (Spark, Briefly)
  - §8 — Course Roadmap
    - Big Data Engineering vs Big Data Analytics
    - Types of analytics (the "four-question" slide)
  - §9 — Five Worked Numerical Examples
    - Worked Example 1 — Bonferroni false-positive count

Worked Example 2 — HDFS chunk math
Worked Example 3 — MapReduce word-count by hand
Worked Example 4 — MapReduce friend-of-friend (mutual friends)
Worked Example 5 — Cluster sizing

§10 — Practice Questions (12)
§11 — Full Worked Answers
§12 — Ending Key Notes (Revision Cards)
§13 — Formula & Concept Reference

Week 01 · Introduction to Big Data Analytics

Why this PDF is short. Week 1 is the lightest topic of the course. The examiner uses it to seed 8–12 marks of conceptual short-answers (the V's, why traditional DBs fail, what HDFS chunks are) plus one canonical MapReduce word-count or HDFS chunk-math problem. Master §1 and the five worked examples below and you have those marks locked in.

§1 — Beginning Key Notes (Study Compass)

These are the eight load-bearing ideas you must walk into the exam owning. Almost every Week-1 short-answer question reduces to applying one of them.

1. **Big Data definition.** Data that is too large, too fast, or too complex for traditional systems to capture, store, manage, or analyze efficiently. Three (or five) V's: **Volume, Velocity, Variety**, and the extended **Veracity, Value**.
2. **Why traditional systems fail.** Single-machine memory/CPU/disk limits, RDBMS rigid schema, batch-only processing, ACID vs scalability trade-off, network & disk I/O bottlenecks.
3. **Bonferroni's principle.** If you look for too many patterns, *random* coincidences will surface as false "significant" findings. The expected number of false positives = (number of tests) × (probability per test). Significance threshold MUST decrease as the number of tests increases.
4. **Distributed computing principles.** (i) Many cheap commodity machines share work; (ii) **data locality** — move compute to data, not data to compute; (iii) design for **fault tolerance** — expect failure as the norm, not the exception.
5. **HDFS basics.** Files are split into **chunks** (default 128 MB). Each chunk replicated **3 times** by default. **NameNode** stores metadata; **DataNodes** store actual blocks. Replication MULTIPLIES storage, but enables fault tolerance and parallel reads.
6. **MapReduce model.** map(k1,v1)→list(k2,v2)\text{map}(k_1, v_1) \to \text{list}(k_2, v_2), then **shuffle/sort by** k2k_2, then reduce(k2,list(v2))→list(k3,v3)\text{reduce}(k_2, \text{list}(v_2)) \to \text{list}(k_3, v_3). Word-count is the canonical example.
7. **Shuffle is the network bottleneck.** Map runs locally on data (cheap). Reduce runs on grouped keys (cheap). The shuffle step physically moves all map outputs across the network — this dominates the cost. Beginners forget this stage.
8. **Course roadmap.** Beyond Hadoop: similarity search, link analysis (PageRank), frequent itemsets, clustering, recommendation, stream processing, dimensionality reduction. Each maps to a real-world data-mining problem.

> **The single biggest exam pattern for Week 1.** A multi-part question of the form: "Define Big Data, list the V's with one example each, explain *why* RDBMS fails, walk through MapReduce word-count on these three lines." Practice that as a 25-mark essay and you have Week 1 in the bag.

§2 — What is Big Data? The Five V's

Working definition (slide-form): *Big Data refers to datasets that are too large, too fast, or too complex for traditional data processing systems to capture, store, manage, or analyze efficiently.*

A sharper one-liner: **Big Data is data that breaks traditional systems in volume, velocity, or variety, and requires new tools and approaches to generate value.**

V1 — Volume (Scale of Data)

The amount of data is too large to store or process on a single machine. Even reading 1 TB from disk at 100 MB/s takes ~2.8 hours — and you still haven't done any computation.

Examples: Netflix user activity logs, Twitter firehose, bank transaction logs, IoT sensor streams.

V2 — Velocity (Speed of Data)

The speed at which new data is generated and must be processed. Traditional databases are *batch-oriented*; Big Data systems must keep up with continuous streams.

Examples: Google Maps traffic updates every second, X/Twitter trending hashtags, real-time fraud detection.

V3 — Variety (Diversity of Sources)

Data arrives in multiple forms — structured (tables), semi-structured (JSON, XML), and unstructured (text, images, video, audio). RDBMS expect a fixed schema; real data does not oblige.

Examples: Tweets + hashtags + images + geolocation in a single stream; Netflix watch history + ratings + thumbnails + free-text reviews.

V4 — Veracity (Uncertainty of Data)

Data is messy, noisy, incomplete, or actively adversarial. Cleaning at scale is itself a Big Data problem.

Examples: Social-media spam and bots, inaccurate GPS readings, sensor errors in IoT, missing or duplicated records.

V5 — Value (Usefulness for Decisions)

Collecting massive data is useless if it cannot be turned into action. Volume × Velocity × Variety only matters if it produces a decision: a recommendation, a fraud alert, a trend.

Examples: Netflix recommendations driving engagement, fraud alerts preventing losses, trend detection for marketing campaigns.

Comparative table — V's vs traditional system limitations

V	What it means	Traditional system limitation	Example
Volume	Size too big	Single machine fails	Netflix 1 TB logs
Velocity	Too fast	Batch DB too slow	Google Maps traffic
Variety	Many data types	RDBMS rigid schema	Twitter posts
Veracity	Messy / noisy	Hard to clean and process	Social media spam
Value	Insights at scale	Too slow / manual	Fraud detection

Why exactly does Excel / RDBMS fail?

System	Failure mode
Excel	Memory limits, single-machine, no concurrency, no streaming, no fault tolerance
RDBMS	Structured schema only, vertical scaling ceiling, batch-oriented, high write contention, fault-tolerance hard

> **EXAM TRAP — "scaling" is not one thing.** *Vertical scaling* = bigger CPU/RAM on one box (easy but expensive, hits a ceiling). *Horizontal scaling* = add more machines (scales indefinitely but needs distributed logic). Big Data systems are horizontal-scale by design. Mention both axes whenever asked about scalability.

---

§3 — Bonferroni's Principle

When you search a huge dataset for "interesting" patterns, you will find some **even if the data is pure random noise**. This is Bonferroni's warning: the more hypotheses you test, the more false positives you must expect by chance alone.

The principle, stated precisely

If you run m statistical tests, each with per-test false-positive probability p under the null hypothesis, then the **expected number of false positives** is

$$\mathbb{E}[\text{false positives}] = m \cdot p$$

To keep the *family-wise* error rate at level α, the **per-test threshold must shrink to** α / m — this is the Bonferroni correction.

> **EXAM TRAP — significance threshold must DECREASE as the number of tests increases.** Many beginners say "more data → more confidence." The opposite is true: more *tests* on the same data dilute confidence. With one billion tests at $p=0.001$ each, you expect a million false positives.

Worked example — the "TIA" terrorist-pattern example (MMDS §1.2.3)

Suppose intelligence analysts hypothesize that two "terrorists" plan an attack by visiting the same hotel on the same day on two different days within a year. They search a database of 10^9 people, each visiting one hotel out of 10^5 on each of 10^3 days.

How many random pairs of people will appear "suspicious"?

- Probability two specific people visit the same hotel on a specific day: $1/10^5$.
- Probability they do this on **two specific days**: $(1/10^5)^2 = 10^{-10}$.
- Number of pairs of people: $\binom{10^9}{2} \approx 10^{18}/2 = 5 \times 10^{17}$.
- Number of pairs of days: $\binom{10^3}{2} \approx 5 \times 10^5$.
- Expected suspicious pairs: $5 \times 10^{17} \times 5 \times 10^5 \times 10^{-10} \approx 2.5 \times 10^{13}$.

Conclusion. Even if there were *zero* real terrorists, you would surface ~2.5×10^{13} "suspicious" pairs by random coincidence. Investigating them all is impossible — the signal drowns in the noise.

Lesson. At Big Data scale, statistical tests must be designed so that the *expected* number of false positives is small *relative to* the number of true positives you hope to find. Otherwise, "patterns" found are almost certainly noise.

---

§4 — Distributed Computing: Why a Single Machine Cannot Cope

A modern server reads sequentially from disk at ~100 MB/s. To read 1 TB takes:

$$\frac{10^{12} \text{ bytes}}{10^8 \text{ bytes/s}} = 10^4 \text{ s} \approx 2.8 \text{ hours}$$

For 1 PB (10^{15} B), one machine needs ~115 days *just to read the data once*. Computation on top of that is hopeless. The fix: parallel reads from many machines.

The four bottlenecks (course slide form)

Bottleneck	Why it matters
Single-machine compute	Finite CPU, RAM, disk; failure = total downtime.
RDBMS scaling	Vertical scaling hits hardware ceiling; horizontal sharding is complex.
ACID vs scalability	Strict ACID is hard to maintain at billions of writes/sec; relax for speed.
Network and disk I/O	Moving 1 TB across the network is slower than computation; I/O dominates.

The Big Data philosophy (the "at-a-glance" slide)

Principle	Intuition
Distributed computing	Many machines share the work → speed and redundancy.
Data locality	Move compute to data → reduce network overhead.
Fault tolerance	Expect failure → design for automatic recovery.
Commodity hardware	Cheap, replaceable parts → scale inexpensively.
Trade-offs	Consistency vs scalability — prioritize what matters for the app.

> **The single most important practical insight.** *Moving data is more expensive than moving computation.* If a chunk of data sits on machine X, send the *code* to machine X rather than the data to wherever the code happens to live. This is **data locality** — the principle behind HDFS + MapReduce co-design.

§5 — Hadoop Distributed File System (HDFS)

HDFS is the distributed storage layer of Hadoop. Designed for streaming reads of huge files on commodity hardware, with built-in fault tolerance.

Core concepts

- **Chunk (block):** A file is split into fixed-size pieces. Default chunk size = **128 MB** (sometimes 64 MB on older clusters; 256 MB on newer ones).
- **Replication factor:** Each chunk is stored on **3 different DataNodes** by default. If one node dies, the other two have the data.
- **NameNode:** A single master that stores the **metadata** — which chunks belong to which file, which DataNodes hold each chunk. NameNode does NOT store the data itself.
- **DataNodes:** Worker machines that store and serve the actual chunks.
- **Data locality:** When a job needs to process a chunk, Hadoop schedules it on a DataNode that already holds a replica of that chunk — no network transfer.

Mini architecture diagram (ASCII)

```
                    +----------------------+
                    |     NameNode         |
                    |  (file → chunk map,  |
                    |   chunk → DataNodes) |
                    +----------+-----------+
                               |
             -----------------+----------------
             |                |                |
     +--------v------+  +------v--------+  +----v---------+
     |  DataNode 1   |  |  DataNode 2   |  |  DataNode 3  | ...
     | chunks: 1,3,5 |  | chunks: 1,2,4 |  | chunks: 2,3,5|
     +---------------+  +---------------+  +--------------+

   File F (300 MB) → chunks {1, 2, 3} of 128 MB / 128 MB / 44 MB,
   each replicated on 3 of the DataNodes (placement chosen by NameNode).
```

Storage cost

Storing a file of size F bytes with replication factor R uses $F \cdot R$ bytes of raw disk across the cluster (plus a small metadata overhead). For $F = 1$ TB and $R = 3$: 3 TB of physical storage.

> **EXAM TRAP — replication MULTIPLIES storage, it does not divide it.** A common mistake is to think 3× replication "splits" the file three ways. No — it stores **three full copies**. Cost ↑ 3×, fault tolerance ↑, parallel read bandwidth ↑.

> **EXAM TRAP — NameNode is a single point of failure (in classic HDFS).** Without secondary NameNode / standby HA, losing the NameNode loses access to the whole cluster's data even though every DataNode is alive. Modern HDFS uses HA NameNodes; mention this if asked.

---

§6 — MapReduce Programming Model

MapReduce is the programming model that made distributed computation accessible. The user writes two pure functions; the framework handles distribution, scheduling, shuffling, and fault tolerance.

The three phases

1. **Map** — applied independently to each input record. $\text{map}: (k_1, v_1) \to \text{list}((k_2, v_2))$ Many map tasks run in parallel, each on a chunk of the input (data-locality scheduled).
2. **Shuffle and sort** — framework regroups all map outputs by key k_2. Output: for each distinct k_2, the list of all values emitted for it. $(k_2, [v_2, v_2', v_2'', \ldots])$ This is the network-heavy phase.

3. **Reduce** — applied per key. reduce:(k2,list(v2))→list((k3,v3)) \text{reduce}: (k_2, \text{list}(v_2)) \;\to\; \text{list}((k_3, v_3)) Output written to HDFS.

Diagram (ASCII)

```
 Input chunks                                        Output
-----------                                         ------
chunk 1 ──► map ──┐
chunk 2 ──► map ──┤  ── shuffle/sort by k2 ──►  reduce ──► part-00000
chunk 3 ──► map ──┤                              reduce ──► part-00001
chunk 4 ──► map ──┘                              reduce ──► part-00002
                                                     ...
 (parallel,                                        (parallel,
  data-local)                                       per-key)
```

Canonical example — word count

Input: a large text file split into chunks. Each map task reads its chunk line by line.

Map. For each word ww in the line, emit (w,1)(w, 1).

```
map(line_id, line):
    for word in line.split():
        emit(word, 1)
```

Shuffle. Framework groups all (w,1)(w, 1) pairs by ww. For each word, the reducer receives (w,[1,1,1,...]) (w, [1, 1, 1, \ldots]).

Reduce. Sum the list.

```
reduce(word, counts):
    emit(word, sum(counts))
```

> **EXAM TRAP — shuffle is the network bottleneck.** Map and reduce are both embarrassingly parallel and run locally. Shuffle physically moves every (k2,v2)(k_2, v_2) pair from the map node to the reducer responsible for that key. On a 1 TB input with billions of intermediate pairs, shuffle dominates wall-clock time. Many beginner answers skip the shuffle step entirely — losing easy marks.

> **EXAM TRAP — combiners are NOT the same as reducers.** A combiner runs on the map side to *partially* aggregate before shuffle (e.g. sum local 11s into a single (w,n)(w, n) pair). It reduces shuffle volume. It is optional and must be associative + commutative.

§7 — Other Tools (Spark, Briefly)

The course later covers Apache Spark in depth; for Week 1, just know the headline.

- **Spark** keeps intermediate data **in memory** between stages, instead of writing to HDFS like MapReduce does between every job. For iterative algorithms (PageRank, k-means, gradient descent) this is **10–100× faster** than Hadoop MapReduce.
- The basic Spark abstraction is the **RDD** (Resilient Distributed Dataset) — a fault-tolerant, partitioned collection. Higher-level APIs: **DataFrames** (schema-aware, SQL-like) and **Datasets**.
- Spark runs on top of HDFS (or S3, etc.) — it replaces the *compute* layer, not the *storage* layer.

Aspect	Hadoop MapReduce	Apache Spark
Intermediate data	Written to HDFS between jobs	Kept in memory (RDDs)
Iterative algos	Slow (re-read disk each pass)	Fast (data stays in RAM)
API	Map + reduce only	RDD, DataFrame, SQL, MLlib, Streaming
Fault tolerance	Re-execute failed task	Lineage graph re-computes lost RDD

§8 — Course Roadmap

The course preview, with each topic mapped to its real-world data-mining problem:

Topic (later weeks)	Real-world question
Frequent itemsets (W2–3)	Which products are bought together? (market-basket → recommendations)
Finding similar items (W4–5)	Near-duplicate documents, plagiarism, similar-user retrieval (LSH).
Stream data processing (W6–7)	Trending hashtags, real-time fraud, count-distinct on infinite streams.
Graph processing — PageRank (W8–9)	Web-search ranking, citation analysis, social-graph influence.
Recommender systems (W10–11)	“Users who watched X also watched Y” — Netflix, Amazon, YouTube.
Dimensionality reduction (W13)	Compress millions of features into a few latent factors (SVD, PCA).
Community detection (W16)	Find clusters in social networks; spectral graph methods.
Hadoop / Spark / NoSQL (Part 2)	Engineering layer — how to actually run all of the above on commodity HW.
RAG, text analytics (Part 3)	LLM applications, healthcare/business intelligence at scale.

Big Data Engineering vs Big Data Analytics

Big Data Engineering	Big Data Analytics
Build and maintain infrastructure	Extract insights from data
Collect, store, prepare data	Build models and predictions
Ensure scalability, reliability, FT	Support decision-making

Types of analytics (the "four-question" slide)

Type	Question	Example
Descriptive	What happened?	Daily active users, total sales last month
Diagnostic	Why did it happen?	Drop in users due to feature change
Predictive	What will happen?	Next month's sales forecast
Prescriptive	What should we do?	Suggest promotions to maximize sales

§9 — Five Worked Numerical Examples

Read each, then close the document and reproduce the arithmetic from scratch on paper. These are the *exact* shape of the small computational questions Week 1 produces.

Worked Example 1 — Bonferroni false-positive count

Problem. A bank screens $N=100{,}000$ customers for a "rare suspicious pattern" that, under the null, occurs with probability $p=10^{-4}$ per customer. The bank also knows that historically only ~5 customers per year actually exhibit the pattern for fraudulent reasons. (a) Expected number of customers flagged purely by chance? (b) Compare to true positives. (c) What significance threshold would keep the family-wise error rate ≤ 0.05?

(a) Expected false positives. $\mathbb{E}=N\cdot p=100{,}000\times 10^{-4}=10$.

(b) Comparison. True positives ≈ 5; false positives ≈ 10. **Two thirds of all flagged customers are noise.** Investigators waste most of their time on non-fraudsters.

(c) Bonferroni-corrected threshold. Per-test threshold $\le \alpha/N=0.05/100{,}000=5\times 10^{-7}$. The original $p=10^{-4}$ is $200\times$ too loose. Either tighten the per-test threshold or shrink N by pre-filtering.

> **Take-away.** At Big Data scale, $m\cdot p$ blows up linearly with m. Always state expected false positives explicitly.

Worked Example 2 — HDFS chunk math

Problem. A 1.4 GB file is stored in HDFS with default chunk size 128 MB and replication factor 3. (a) How many chunks? (b) How much physical disk space across the cluster? (c) If chunk size were 64 MB instead, how many chunks?

(a) Number of chunks. $1.4\text{ GB} = 1.4 \times 1024\text{ MB} \approx 1433.6\text{ MB}$. Number of full 128 MB chunks $= \lfloor 1433.6/128 \rfloor = 11$ full chunks of 128 MB = 1408 MB; remainder $1433.6 - 1408 = 25.6$ MB occupies a 12th (partial) chunk. **Total = 12 chunks.**

(b) Total disk usage. Each chunk replicated 3×: $12 \times 3 = 36$ blocks across the cluster. Raw bytes stored $\approx 1.4\text{ GB} \times 3 = 4.2\text{ GB}$.

(c) With 64 MB chunks. $\lceil 1433.6/64 \rceil = \lceil 22.4 \rceil = 23$ chunks. Replicated 3× → 69 blocks.

> **EXAM TIP.** State the formula: *chunks* $= \lceil F/B \rceil$, *blocks on disk* $= R \cdot \lceil F/B \rceil$. Then plug in numbers. Examiners reward the formula even if your arithmetic slips.

Worked Example 3 — MapReduce word-count by hand

Problem. Three input lines (one per chunk for clarity):

```
line 1: hello world
line 2: hello data
line 3: world data analytics
```

Walk through map, shuffle, and reduce. Produce the final word-frequency table.

Step 1 — Map outputs (per line).

Line	Map emits
1	(hello, 1), (world, 1)
2	(hello, 1), (data, 1)
3	(world, 1), (data, 1), (analytics, 1)

Step 2 — Shuffle / group by key.

Key	Value list
hello	[1, 1]
world	[1, 1]
data	[1, 1]
analytics	[1]

Step 3 — Reduce (sum the list).

Word	Count
hello	2
world	2
data	2
analytics	1

Optional combiner. A combiner on each map output would pre-aggregate within a single mapper. Useful when each chunk has many repeated words; pointless on these tiny lines.

> **EXAM PATTERN.** Always show three columns: map output → shuffle group → reduce output. Skipping shuffle is the classic mistake — examiners deduct marks even if the final answer is correct.

Worked Example 4 — MapReduce friend-of-friend (mutual friends)

Problem. A small social graph as an adjacency list (undirected):

```
 A: [B, C, D]
B: [A, C, E]
C: [A, B, D]
D: [A, C]
E: [B]
```

Design a MapReduce job that, for every pair of users (u,v)(u, v) who are *both* friends with at least one common person, outputs the count of mutual friends. Show the map and reduce key/value design.

Map. For each user uu with friend list LL, for every unordered pair (v1,v2)(v_1, v_2) in LL with v1<v2v_1 < v_2, emit:

key=(v1,v2),value=u \text{key} = (v_1, v_2), \quad \text{value} = u

i.e. "v1v_1 and v2v_2 share a mutual friend uu".

For example, A's list [B,C,D][B, C, D] produces: ((B,C),A)((B,C), A), ((B,D),A)((B,D), A), ((C,D),A)((C,D), A).

All map outputs across the graph:

Source u	Pairs emitted (as key → value)
A	(B,C)→A, (B,D)→A, (C,D)→A
B	(A,C)→B, (A,E)→B, (C,E)→B
C	(A,B)→C, (A,D)→C, (B,D)→C
D	(A,C)→D
E	(no pair — only one friend)

Shuffle — group by key.

Pair	Mutual friends list
(A,B)	[C]
(A,C)	[B, D]
(A,D)	[C]
(A,E)	[B]
(B,C)	[A]
(B,D)	[A, C]
(C,D)	[A]
(C,E)	[B]

Reduce. Output `(pair, len(list))`.

Pair	#Mutual friends
(A,C)	2
(B,D)	2
All others	1

> **Why this design works.** The key choice (v1,v2)(v_1, v_2) groups together every "vote" that v1v_1 and v2v_2 are linked by a common friend. The reducer just counts. This pattern — "emit per pair, reduce per pair" — generalizes to many graph-mining problems.

Worked Example 5 — Cluster sizing

Problem. Process 1 PB (101510^{15} bytes) of data in 1 hour. Each commodity machine reads at 100 MB/s (10810^8 bytes/s). Roughly how many machines are needed (lower bound, ignoring shuffle and overheads)?

Per-machine throughput in 1 hour: 108 B/s×3600 s=3.6×101110^8 \text{ B/s} \times 3600 \text{ s} = 3.6 \times 10^{11} B = 360 GB.

Number of machines needed:

10153.6×1011≈2778 machines \frac{10^{15}}{3.6 \times 10^{11}} \approx 2778 \text{ machines}

Add headroom. Real workloads spend perhaps half their time in shuffle, GC, retries. Multiply by ~2: budget ~5000–6000 machines. Add replication (3×) and you have ~9000 disks.

Sanity check. A single machine would need 1015/108=10710^{15} / 10^8 = 10^7 s ≈ 116 days *just to read* the data. Distribution is non-negotiable.

> **EXAM TIP.** Always show the formula nodes=total bytesper-node B/s×time\text{nodes} = \frac{\text{total bytes}}{\text{per-node B/s} \times \text{time}}, then plug numbers. State your assumptions (sequential read, no shuffle overhead) explicitly.

§10 — Practice Questions (12)

Mix: 4 numerical, 5 conceptual short-answer, 3 MCQ / true-false. Time yourself: ~5 min per numerical, ~3 min per conceptual, ~1 min per MCQ.

Q1 [Numerical · 4 marks]. A 600 MB log file is stored in HDFS with default chunk size 128 MB and replication factor 3. (a) How many chunks? (b) How much total physical disk space is consumed across the cluster?

Q2 [Numerical · 5 marks]. A retailer screens N=50,000N = 50{,}000 daily transactions for a fraud signature with per-test false-positive probability p=2×10−3p = 2 \times 10^{-3}. (a) Expected number of false-positive flags per day. (b) If true fraud rate is 30 transactions per day, what fraction of flagged transactions are real fraud? (c) Bonferroni-corrected per-test threshold for family-wise α=0.01\alpha = 0.01.

Q3 [Numerical · 6 marks]. Walk through a MapReduce word count on:

```
 line 1: big data is big
line 2: data is fun
```

Show map outputs per line, the shuffle grouping, and the final reduce output.

Q4 [Numerical · 5 marks]. You must process 500 TB of input in 30 minutes. Each node reads at 200 MB/s. (a) How many nodes are required (lower bound)? (b) Suppose shuffle and overheads consume 60 % of wall-clock time — how many nodes now?

Q5 [Concept · 3 marks]. State the 5 V's of Big Data and give one *concrete* real-world example for each.

Q6 [Concept · 4 marks]. Explain why a relational DBMS ("traditional RDBMS") cannot handle a Twitter-firehose-style workload. Cite at least three specific limitations.

Q7 [Concept · 4 marks]. State Bonferroni's principle in your own words. Why does the "significance threshold" need to *decrease* as the number of tests grows?

Q8 [Concept · 4 marks]. Describe the four roles in HDFS: (a) chunk, (b) replication factor, (c) NameNode, (d) DataNode. Why does HDFS *replicate* every chunk three times instead of doing RAID-style parity within one machine?

Q9 [Concept · 4 marks]. Define **data locality**. Why does it matter for performance? Tie your answer to the bandwidth gap between local disk reads and inter-rack network transfers.

Q10 [MCQ · 2 marks]. Which statement about MapReduce is TRUE?

1. Map and reduce run in parallel on the same machines as the input chunks.
2. The shuffle phase moves all map outputs across the network, grouped by key.
3. The reduce phase always produces exactly one output file per input chunk.

4. Map functions must be stateful to track partial sums.

Q11 [MCQ · 2 marks]. Default HDFS chunk size and default replication factor in a stock Hadoop install:

1. 64 MB and 2
2. 128 MB and 3
3. 256 MB and 1
4. 512 MB and 3

Q12 [True/False · 2 marks]. "Replication factor 3 in HDFS reduces total storage cost by a factor of 3 because data is split across 3 nodes." Justify.

---

§11 — Full Worked Answers

A1. (a) Chunks $=\lceil 600/128 \rceil = \lceil 4.6875 \rceil = 5$. (b) Disk space $=5 \text{ chunks} \times 128 \text{ MB} \times 3 \text{ replicas} = 1920$ MB ≈ 1.875 GB. *(Note: the last chunk holds only 88 MB of actual data, but is still allocated as a full block in some accounting models. If your course counts only actual bytes:* $600 \text{ MB} \times 3 = 1800$ *MB. State your assumption explicitly.)*

A2. (a) $\mathbb{E}[\text{FP}] = N p = 50{,}000 \times 2 \times 10^{-3} = 100$ false-positive flags per day. (b) Total flags ≈ 100 (FP) + 30 (TP) = 130. Real fraud fraction $=30/130 \approx 23\%$. Investigators waste 77 % of their effort. (c) Bonferroni threshold $=\alpha/N = 0.01/50{,}000 = 2 \times 10^{-7}$. The current threshold $p = 2 \times 10^{-3}$ is $10{,}000 \times$ too loose.

A3.

Map outputs:

Line	Pairs
1	(big,1), (data,1), (is,1), (big,1)
2	(data,1), (is,1), (fun,1)

Shuffle:

Key	Values
big	[1, 1]
data	[1, 1]
is	[1, 1]
fun	[1]

Reduce output:

Word	Count
big	2
data	2
is	2
fun	1

A4. (a) Per-node throughput in 30 min =200 MB/s×1800 s=3.6×105 MB=360= 200 \text{ MB/s} \times 1800 \text{ s} = 3.6 \times 10^5 \text{ MB} = 360 GB. Total bytes =500 TB=5×1014= 500 \text{ TB} = 5 \times 10^{14} B = 5×1085 \times 10^8 MB. Nodes =5×108/3.6×105≈1389= 5 \times 10^8 / 3.6 \times 10^5 \approx 1389 nodes. (b) Effective throughput is only 40 % of nominal: divide nodes by 0.4 → ~3475 nodes. Round up and budget ~3500 machines.

A5. Volume — Netflix watch logs (TBs/day). **Velocity** — Google Maps GPS pings (millions/sec). **Variety** — Twitter (text + images + geolocation + JSON metadata). **Veracity** — IoT sensor readings (drift, missing values, spam). **Value** — fraud-detection alerts that prevent dollar losses.

A6. RDBMS fails for Twitter-firehose because (i) **rigid schema** — tweets have heterogeneous fields (text, hashtags, geo, media URLs) that don't fit one table cleanly; (ii) **vertical scaling ceiling** — even a top-end server cannot ingest millions of writes/sec; (iii) **ACID overhead** — strict transactional guarantees serialize writes, killing throughput; (iv) **no native streaming** — RDBMS is batch-oriented; trends need second-level latency; (v) **no fault tolerance for a single big box** — one machine failure = total downtime.

A7. Bonferroni's principle: when running m statistical tests at per-test false-positive probability p, the *expected* number of random false positives is mp. As m grows (Big Data scale), mp explodes — most "significant" findings are noise. To control the family-wise error rate at level α, lower the per-test threshold to α/m. The threshold *must* decrease because every additional test gives noise another chance to surface as a "discovery".

A8. Chunk — fixed-size piece of a file (default 128 MB) — the unit of storage and parallelism. **Replication factor** — number of copies of each chunk stored on distinct DataNodes (default 3). **NameNode** — master holding metadata: file → chunk list, chunk → DataNode list. Does NOT store data. **DataNode** — worker holding actual chunk bytes and serving them. *Why replication, not RAID?* RAID protects against disk failures *within* one machine. HDFS-scale failures are whole-machine and whole-rack. Replicating across machines (and racks) tolerates *node* failure, gives parallel-read bandwidth (multiple readers can read the same chunk from different nodes), and enables data-locality scheduling (any of the 3 replicas is a valid local target).

A9. Data locality = scheduling computation on a node that already holds (a replica of) the data, so no network transfer is needed. It matters because local sequential disk read ≈ 100–500 MB/s, but inter-rack network ≈ 10–50 MB/s effective per stream. Moving a 128 MB chunk across the network is 3–10× slower than reading it locally. At the cluster scale, data locality is the dominant determinant of throughput. Hadoop's scheduler explicitly tries node-local → rack-local → off-rack placement in that order.

A10. (B). (A) is partly true (map runs locally) but reduce runs on different nodes after shuffle. (C) wrong — number of reducers is configurable, independent of input chunks. (D) wrong — map is intentionally stateless for parallelism.

A11. (B) — 128 MB chunks, replication 3.

A12. False. Replication factor 3 *triples* total storage cost (every chunk has 3 full copies). The benefit is fault tolerance, parallel-read bandwidth, and data locality, NOT storage savings.

---

§12 — Ending Key Notes (Revision Cards)

Term	Quick-fact
Big Data definition	Too large, too fast, or too complex for traditional systems to handle efficiently.
Volume	Scale of data — Netflix logs, Twitter firehose, bank transactions.
Velocity	Speed of generation — Google Maps GPS pings, real-time fraud.
Variety	Multiple forms — structured + semi-structured + unstructured.
Veracity	Uncertainty / noise — spam, sensor errors, missing values.
Value	Usefulness for decisions — recommendations, fraud alerts.
Vertical scaling	Bigger CPU / RAM on one box. Easy but ceilings.
Horizontal scaling	More machines. Scales indefinitely; needs distributed logic.
Bonferroni's principle	$\mathbb{E}[FP]=m\cdot p$. Threshold must shrink to α/m.
Data locality	Move compute to data, not data to compute.
HDFS chunk	Default 128 MB. Unit of storage + parallelism.
Replication factor	Default 3. MULTIPLIES storage; enables fault tolerance + parallel reads.
NameNode	Master, metadata only. Single-point-of-failure in classic HDFS.
DataNode	Worker, holds and serves chunk bytes.
Map function	$map(k1,v1)\to list(k2,v2)$. Stateless, parallel.
Shuffle	Group all $(k2,v2)$ by $k2$. Network-heavy bottleneck.
Reduce function	$reduce(k2,[v2,\ldots])\to list(k3,v3)$. Per-key.
Combiner	Map-side partial reduce. Cuts shuffle volume. Must be associative + commutative.
Spark vs MapReduce	Spark keeps intermediates in RAM → 10–100× faster on iterative algos.
Mistake — replication	3× = 3 full copies. NOT split-three-ways.
Mistake — shuffle skipped	Always show map → shuffle → reduce. Marks lost otherwise.
Mistake — V's example	Don't say "lots of data" for Volume. Cite a concrete TB-scale source.

§13 — Formula & Concept Reference

Concept	Formula / Statement	When to use
Bonferroni expected FP	$\mathbb{E}[FP]=m\cdot p$	Estimating noise floor before running many tests.
Bonferroni-corrected threshold	per-test $\alpha'=\alpha/m$	Setting significance for multiple-comparison studies.
HDFS chunk count	$\#chunks=\lceil F/B\rceil$ (F = file, B = chunk size)	Sizing storage and number of map tasks.
HDFS total disk usage	$\#blocks=R\cdot\lceil F/B\rceil$, bytes $=F\cdot R$	Capacity planning across the cluster.
Sequential read time	$T=F/B_{disk}$ (B_{disk} = disk read MB/s)	Justifying need for distribution.
Cluster sizing (lower bound)	$\#nodes=\text{total bytes}/\text{per-node B/s}\cdot\text{time}$	Back-of-envelope hardware budget.
MapReduce model	$map:(k_1,v_1)\to list(k_2,v_2)$; shuffle by k_2; $reduce:(k_2,[v_2])\to list(k_3,v_3)$	Designing any MR job.
Word-count map	`for word in line: emit(word, 1)`	Canonical map function.
Word-count reduce	`emit(word, sum(counts))`	Canonical reduce function.

Connections to other weeks: - **W2–3 — Frequent Itemsets:** A-priori on top of MapReduce. Bonferroni reappears when justifying minimum support thresholds. - **W4–5 — Finding Similar Items:** LSH / MinHash on large document collections; same map → group → reduce shape. - **W8–9 — PageRank:** matrix–vector product as a MapReduce job (MMDS §5.2.2). The shuffle pattern is identical to word count. - **Part 2 — Hadoop / Spark:** the *engineering* view of everything in §4–§6 above.

One-line summary of Week 1. Big Data is data that breaks single-machine systems; we cope by distributing storage (HDFS — chunks + replication) and computation (MapReduce — map, shuffle, reduce); we must remain statistically careful, because at scale, noise looks like signal (Bonferroni).

End of W01 Introduction to Big Data Analytics exam-prep document.